

OBJETIVOS

- Familiarizar o aluno com o SO MINIX e testes em ambiente de máquina virtual.
 - Compilar e modificar o kernel
- Criação de processos, IPC, gerenciamento de sinais
 - Processos órfãos e zumbis
- Gerenciamento de I/O no MINIX
 - Criar uma chamada de sistema;
 - Criar uma chamada de *kernel* e usá-la via uma chamada de sistemas (item anterior);
 - Criar um *character device*.

TAREFA 3: RECOMPILANDO O KERNEL DO MINIX (de 05/02/2024 a 08/02/2024)

Nesta atividade você irá recompilar o *kernel* do MINIX. Para tal, vá para o diretório `/usr/src/tools` e chame o comando `make`. Verifique as opções que aparecem no arquivo `Makefile` e descubra como um kernel é compilado e instalado.

Nota: uma vez instalado o kernel e reiniciado o MINIX, é importante selecionar a segunda opção “*Start custom kernel*” na tela de início. De outra forma, o MINIX não irá executar o kernel mais recentemente compilado.

Estando apto a construir um novo kernel, finalmente é hora de fazer uma pequena modificação no SO. Primeiro, entretanto, você deve fazer uma cópia “limpa” da árvore de diretórios do kernel. Isto é necessário por dois motivos. Primeiro, quando posteriormente você fizer mudanças que não funcionem e quando precisar de uma referência para consultar, a árvore com os fontes originais estarão lá (percebe agora, por que fazer uma instalação de 2 GB?). Segundo, e mais importante, tal referência é necessária para que você possa criar *patches* (veja abaixo). Assim, faça “`cp -r /usr/src/ /usr/src.clean`”, e você pode continuar.

Dica: para evitar erros ou impossibilitar o uso de seu MINIX, você também pode usar o recurso de

snapshot do VirtualBox.

Para esta prática, implementaremos uma pequena modificação no *kernel*. Você deve **imprimir o nome de cada arquivo que está sendo executado pelo SO**. Para tanto, você deve localizar os fontes do *kernel* que implementam a chamada de sistema `exec`. Então, no ponto correto, você deve inserir uma chamada `printf` que coloque a string “executando” seguida pelo nome do arquivo que está sendo executado. Por exemplo: quando você digita “`ls`” no shell, você deve ver a string “MSG: executando `ls`” impressa no console. Isto deve em verdade, requerer apenas uma única linha a ser adicionada, mas isto deve mostrar algumas características de depuração (*debug*) muito simples a você.

Uma vez que você faça essa modificação, recompile o *kernel*, instale-o e reinicie o sistema. Caso você seja bem sucedido, você deverá poder ver uma série de mensagens que permitam que você veja quais programas o sistema operacional está executando (*launching*).

Dica: leia o artigo “How to add a system call and System call sequence for MINIX3” (disponível na área de material didático) para entender melhor como as chamadas de sistema são servidas no SO MINIX. Uma vez que você entenda o mecanismo, você pode facilmente localizar o lugar para inserção do código.

Sua última tarefa é submeter sua mudança ao professor. Para lidar com grandes sistemas de software (e.g. sistemas operacionais como o MINIX ou o Linux), não faz sentido enviar o código fonte completo para cada pequena mudança. Para isto, foram inventados os *patches*. Um *patch* captura as mudanças entre dois arquivos diferentes (arquivo antigo e arquivo novo). Dado o arquivo antigo e o *patch*, é possível usar o programa `patch` para criar o arquivo novo. É exatamente isso que você deve fazer. Dessa forma, ao terminar a sua modificação do kernel (incluindo recompilar, instalar e testar tudo), você deve ir ao diretório **/usr** e produzir um *patch* para cada arquivo que você modificou. Para isto, você deve chamar o comando `diff -c (path/to/oldfile) (path/to/newfile)` e armazenar a saída em um arquivo chamado `seunome.patch`. Por exemplo, ao fazer um *patch* do arquivo `/usr/src/servers/inet/clock.c`, você deve primeiro mudar para o diretório **/usr** e então executar o comando `diff -c src/clean/servers/inet/clock.c`

`src/servers/inet/clock.c`. Você pode testar se o seu *patch* funcionou pela criação de outra cópia da árvore de diretórios (e.g. `/usr/test`), ir ao diretório `/usr/test` e executar o comando `patch -p1 < seunome.patch`. Isto deve modificar o arquivo `clock.c` no diretório `/usr/test` da maneira esperada. Além disso, é assim que os *patches* dos alunos serão testados. Espera-se que eles possam ser aplicados de maneira correta (sem quaisquer erros ou *warnings*).

Dica: use redirecionamento de saída. Dessa forma é possível escrever a saída do comando `diff` diretamente em um arquivo texto.